

Angular 19

Introducao simples para estudo e consulta rapida

Objetivo	Entender a estrutura basica de uma aplicacao Angular moderna usando componentes standalone, signals, servicos e rotas.
Nivel	Iniciante a intermediario
Tecnologias	Angular 19, TypeScript, HTML e CSS

Angular e um framework para construir aplicacoes web organizadas em componentes. A versao 19 segue a abordagem moderna da plataforma, com forte uso de componentes standalone, injeção de dependencias, reatividade com signals e ferramentas integradas para rotas, formularios e requisicoes HTTP.

Ideia central: a tela e dividida em componentes pequenos. Cada componente possui logica em TypeScript, estrutura em HTML e estilo em CSS.

1. Criando um projeto

Com Node.js instalado, o Angular CLI pode criar e executar o projeto.

```
npm install -g @angular/cli
ng new meu-app
cd meu-app
ng serve
```

Depois, acesse <http://localhost:4200>. O comando `ng serve` inicia um servidor de desenvolvimento e recompila a aplicação quando os arquivos são alterados.

2. Componente standalone

Um componente representa uma parte da interface. Em um projeto moderno, ele pode funcionar sem precisar ser declarado manualmente em um `NgModule`.

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  template: `
    <h1>{{ titulo }}</h1>
    <button (click)="alterarTitulo()">Alterar</button>
  `,
})
export class AppComponent {
  titulo = 'Meu primeiro app Angular 19';

  alterarTitulo() {
    this.titulo = 'Titulo atualizado!';
  }
}
```

`{{ titulo }}` faz interpolação de dados. O evento `(click)` chama um método da classe quando o botão é pressionado.

3. Estado reativo com signals

Signals guardam valores reativos. Quando um signal muda, o Angular atualiza os pontos da interface que dependem dele.

```
import { Component, signal } from '@angular/core';

@Component({
  selector: 'app-contador',
  standalone: true,
  template: `
    <p>Contagem: {{ contador() }}</p>
    <button (click)="incrementar()">Somar</button>
  `,
})
export class ContadorComponent {
  contador = signal(0);

  incrementar() {
    this.contador.update(valor => valor + 1);
  }
}
```

4. Servicos e injeção de dependências

Servicos concentram logica reutilizavel, como acesso a API, regras de negocio ou armazenamento de dados. O Angular cria e fornece esses objetos por injeção de dependências.

```
import { Injectable } from '@angular/core';

@Injectable({ providedIn: 'root' })
export class UsuarioService {
  obterNome() {
    return 'Ana';
  }
}

import { Component, inject } from '@angular/core';
import { UsuarioService } from './usuario.service';

@Component({
  selector: 'app-perfil',
  standalone: true,
  template: `<p>Usuario: {{ nome }}</p>`,
})
export class PerfilComponent {
  private usuarioService = inject(UsuarioService);
  nome = this.usuarioService.obterNome();
}
```

5. Rotas

O roteamento permite trocar de pagina sem recarregar todo o site.

```
import { Routes } from '@angular/router';
import { HomeComponent } from './home.component';
import { SobreComponent } from './sobre.component';

export const routes: Routes = [
  { path: '', component: HomeComponent },
  { path: 'sobre', component: SobreComponent },
];
```

6. Boas praticas para um projeto pequeno

- Separe componentes por responsabilidade.
- Use servicos para logica compartilhada e chamadas HTTP.
- Evite colocar regras complexas diretamente no template.
- Use signals para estados locais simples e reativos.
- Crie nomes claros para arquivos, componentes e variaveis.
- Mantenha chaves e URLs sensiveis fora do codigo-fonte.

Resumo: Angular 19 permite construir interfaces por componentes, organizar logica em servicos, reagir a mudancas de estado com signals e navegar entre telas com o roteador. O melhor caminho para aprender e criar uma aplicacao pequena e evolui-la por etapas.